A Comprehensive Bachelor-Level Curriculum for AI Systems Engineering and MLOps

The paradigm of artificial intelligence has irrevocably shifted from the isolated laboratories of academic research into the demanding, high-stakes arena of enterprise software operations. This transition has illuminated a severe industry deficit: a lack of engineers capable of bridging the gap between raw machine learning models and highly available, distributed production systems. The role of the AI Systems Engineer, often overlapping with the Machine Learning Operations (MLOps) Specialist, has emerged to fulfill this exact need. This discipline demands a hybrid skill set that heavily indexes on infrastructure provisioning, continuous integration, API orchestration, and high-performance model serving, rather than the deep mathematical theorems associated with traditional data science.

The following curriculum is architected to provide an exhaustive, four-year bachelor-level equivalent education tailored specifically for an independent, self-directed practitioner. Recognizing the baseline competencies already established—comfort with Linux (Ubuntu) environments, familiarity with command-line tools, operational experience with home servers (such as Unraid), and practical exposure to running local Large Language Models (LLMs) and multi-agent systems—this curriculum aggressively bypasses introductory computing concepts. Furthermore, in strict adherence to the specified constraints, the curriculum relies entirely on free, open-source educational resources, documentation, and tools. Heavy theoretical mathematics has been deliberately omitted in favor of conceptual, code-driven learning that leverages Python primarily as an integration and scripting layer.

The objective of this curriculum is absolute job readiness. By progressing sequentially through eight rigorously defined semesters, the practitioner will transform their hobbyist knowledge into enterprise-grade systems engineering capability, ready to deploy, orchestrate, and manage sophisticated AI architectures across both cloud and bare-metal environments.

Semester 1: Advanced Infrastructure Automation and Data Engineering Foundations

The foundation of any robust artificial intelligence system is not the neural network itself, but the data pipeline that feeds it and the infrastructure upon which it resides. A practitioner already comfortable with operating an Unraid home server must elevate those manual configuration skills into the realm of enterprise Infrastructure as Code (IaC) and distributed data processing. The primary objective of the first semester is to establish absolute programmatic control over infrastructure and master the lifecycle of raw data.

The curriculum commences with "The Missing Semester of Your CS Education," a course provided by the Massachusetts Institute of Technology (MIT). While a practitioner familiar with

Linux will recognize some concepts, this course addresses the critical, often-overlooked mechanics of advanced command-line proficiency, robust shell scripting, code profiling, and debugging. The mastery of advanced Git workflows and Vim is required to ensure the engineer can operate efficiently in headless server environments without reliance on graphical interfaces. The curriculum folds AI-enabled tools into these workflows, teaching the practitioner how to utilize modern automation while remaining aware of its shortcomings.
Following this optimization of personal development tooling, the curriculum shifts to the Data Engineering Zoomcamp, a highly regarded, free nine-week intensive course provided by DataTalksClub. This course transitions the engineer from manual server deployments to fully automated, reproducible architectures. Module 1 introduces containerization utilizing Docker and PostgreSQL, while immediately enforcing the use of Terraform for declarative infrastructure provisioning on platforms like Google Cloud Platform (GCP). The ability to containerize applications and provision resources via Terraform guarantees that environments are identical across development, staging, and production—a non-negotiable requirement for MLOps.
Data pipelines must be scheduled, monitored, and capable of recovering from failures. Module 2 of the Zoomcamp introduces workflow orchestration utilizing Kestra, a powerful engine for building data pipelines that can schedule, retry, and backfill massive datasets. The curriculum notes that the tooling ecosystem evolves rapidly; for instance, Kestra replaced Mage in the 2025 cohort, reflecting the industry's shift toward highly available orchestration layers.
The remainder of the semester focuses on the transformation and processing of data at scale. Modules 3 and 4 dive deep into Data Warehousing utilizing BigQuery, teaching advanced concepts such as table partitioning, data clustering, and query cost optimization. This is paired with Analytics Engineering utilizing dbt (data build tool), allowing the engineer to write modular SQL transformations that build staging, intermediate, and data mart models. Finally, Modules 5 and 6 introduce distributed processing paradigms. The engineer will utilize Apache Spark for massive batch processing jobs and Apache Kafka paired with Apache Flink for real-time streaming data ingestion and schema serialization.

Semester 1 Curriculum Block	**Technical Focus**	**Core Technologies**	**Primary Resource**
Advanced OS Tooling	Shell scripting, profiling, version control workflows	Bash, Git, Vim	MIT Missing Semester
Infrastructure as Code	Containerization, cloud provisioning	Docker, Terraform, GCP	Data Eng Zoomcamp (Mod 1)
Workflow Orchestration	Pipeline scheduling, backfilling, state management	Kestra	Data Eng Zoomcamp (Mod 2)

Data Warehousing	Table partitioning, cost optimization, transformations	BigQuery, dbt	Data Eng Zoomcamp (Mod 3-4)
Distributed Processing	High-throughput batch and streaming pipelines	Apache Spark, Kafka, Flink	Data Eng Zoomcamp (Mod 5-6)

The capstone project for Semester 1 involves the construction of a fully automated, production-ready data ingestion pipeline deployed on the practitioner's local Unraid server. The engineer must write Terraform scripts to provision an isolated network containing a PostgreSQL database and a Kestra orchestration engine. They must then develop a Python extraction script that pulls real-time data from a public API, containerize this script with Docker, and orchestrate its execution via Kestra. The data must then be transformed within the database utilizing dbt, culminating in a pristine dataset ready for machine learning consumption. This project demonstrates foundational mastery of the infrastructure primitives required for AI engineering.

Semester 2: Backend Architecture and Microservices API Development

With a fortified understanding of data engineering and infrastructure, the curriculum advances to the development of the programmatic interfaces that allow disparate software systems to interact with machine learning models. AI models do not exist in a vacuum; they must be securely exposed to frontend applications, mobile devices, and other backend services. Semester 2 focuses entirely on mastering Python-based Application Programming Interfaces (APIs) and the architectural patterns necessary to support high-latency machine learning inference.

The language of choice for this integration work is Python. The curriculum leverages the freeCodeCamp comprehensive course on FastAPI, a modern, highly performant web framework for building APIs. FastAPI is uniquely suited for AI systems engineering due to its native support for asynchronous programming and automatic data validation via Python type hints and Pydantic models.

The instructional progression demands a thorough mastery of core web protocols. The engineer will learn to define and validate path parameters, query parameters, and complex JSON request bodies. They must become fluent in the semantic usage of HTTP methods, utilizing POST for data submission, PUT for updating existing records, and DELETE for resource removal. The curriculum emphasizes the integration of these APIs with NoSQL databases, specifically utilizing the PyMongo library to interface with MongoDB for the rapid storage and retrieval of dynamic document structures.

However, building a monolithic API is insufficient for machine learning workloads. When an API

endpoint receives a request that requires a massive neural network to generate a response, the computational time can range from hundreds of milliseconds to several seconds. If the API is strictly synchronous, the web server will block, leading to timeouts and degraded performance. Therefore, the curriculum mandates a deep dive into microservice architectures and asynchronous job queuing.

The practitioner will study the implementation of decentralized architectures using Redis and Redis Streams. By decoupling the user-facing web server from the heavy inference engine, the system becomes highly resilient. The engineer must learn to implement background tasks, allowing the FastAPI server to immediately acknowledge a user's request, place the computational job into a Redis stream, and return a tracking ID while a separate worker service processes the heavy ML workload asynchronously.

Semester 2 Curriculum Block	**Technical Focus**	**Core Technologies**	**Primary Resource**
API Fundamentals	Routing, path/query parameters, request validation	Python, FastAPI, Pydantic	freeCodeCamp FastAPI Crash Course
Database Integration	NoSQL document storage, connection pooling	MongoDB, PyMongo	freeCodeCamp FastAPI Quickstart
Asynchronous Design	Non-blocking I/O, background tasks	Python asyncio	freeCodeCamp Microservices Guide
Microservices	Decoupled architecture, message brokering	Redis, Redis Streams	freeCodeCamp Microservices Guide

The capstone project for Semester 2 is the development of an asynchronous, microservice-based AI Gateway. The architecture requires three distinct containerized services deployed via Docker Compose. The first service is a FastAPI web server that accepts text inputs from a user, validates the payload using Pydantic, writes a "pending" record to a MongoDB instance, and pushes a task message into a Redis queue. The second service is a Python worker script that continuously polls the Redis queue, simulates a long-running machine learning inference task using asyncio.sleep, and subsequently updates the MongoDB record with the "completed" result. The client application can then use the tracking ID to poll the API and retrieve the final output. This asynchronous pattern perfectly mimics the robust deployment architectures utilized by leading AI platforms.

Semester 3: Machine Learning Systems Design and Data Lifecycles

Transitioning from traditional software engineering into AI systems engineering requires a fundamental paradigm shift. Traditional software operates on deterministic logic: defined inputs always produce expected outputs based on rigid rules. Machine learning systems, conversely, are inherently probabilistic and dynamic. Semester 3 abstracts away the algorithmic code to focus entirely on the holistic design of machine learning systems, emphasizing the complete lifecycle of a model from conception through continuous production monitoring.
The curriculum utilizes Stanford's CS329S: Machine Learning Systems Design, developed by Chip Huyen, augmented by the Full Stack Deep Learning (FSDL) course. The engineer must internalize the critical dichotomy between machine learning in research versus machine learning in production. In academic research, the primary focus is isolated model performance (accuracy, F1 score), utilizing static datasets with a priority on fast training times and high computational throughput. In a production environment, the constraints are radically different. Production systems serve multiple stakeholders, are severely constrained by inference latency, and must generate rapid predictions against noisy, unpredictable, real-world data streams.
The curriculum mandates a deep exploration of data management within ML systems. The engineer will study the challenges of sourcing, cleaning, augmenting, and synthesizing real-world data. Understanding the different layers of the data pipeline—including data processors, monitors, and feature stores—is crucial, as data engineering and annotation typically consume the vast majority of an ML project's lifecycle.
Furthermore, the engineer must master the diagnosis of ML system failures. Unlike traditional software that crashes loudly with a stack trace, machine learning systems often fail silently by generating increasingly inaccurate predictions. This is typically caused by data distribution shifts—the phenomenon where the statistical properties of the incoming production data slowly diverge from the data upon which the model was originally trained. The practitioner will study monitoring strategies and the principles of continual learning, designing feedback loops that automatically detect concept drift and trigger retraining pipelines. The curriculum also integrates vital discussions on the ethical implications, privacy concerns, and security vulnerabilities inherent in deploying AI systems.

Semester 3 Curriculum Block	**Technical Focus**	**Core Concepts**	**Primary Resource**
System Trade-offs	Research vs. Production, Latency vs. Throughput	System Design Objectives	Stanford CS329S
Data Management	Sourcing, synthesis, feature stores	Data Pipeline Architecture	Stanford CS329S / FSDL

Failure Diagnosis	Silent failures, data distribution shifts	Concept Drift, Covariate Shift	Stanford CS329S
Continual Learning	Automated retraining, feedback loops	Model Monitoring	Full Stack Deep Learning (FSDL)

The Semester 3 capstone project deviates from coding into pure architectural systems design. The engineer must author a comprehensive, professional-grade System Architecture Document for a proposed high-frequency, real-time AI system (such as a dynamic pricing engine or an automated fraud detection network). The document must explicitly define the data ingestion pipelines, the feature store architecture, the strict latency constraints for the inference engine, and the statistical methods used for continuous monitoring. The practitioner must map out the exact points in the architecture where data distribution shifts are most likely to occur and define the automated CI/CD triggers required to initiate model retraining. This exercise solidifies the system-level thinking required to orchestrate complex AI operations.

Semester 4: Neural Network Architectures and Conceptual Deep Learning

While an MLOps specialist is primarily concerned with infrastructure and deployment pipelines, treating machine learning models as impenetrable black boxes is a critical error. To optimize inference engines, debug memory leaks during serving, and select appropriate hardware architectures (CPU vs. GPU vs. TPU), the systems engineer must understand the internal mechanics of neural networks. Semester 4 provides this mathematical and architectural intuition. Crucially, addressing the user's specific constraints, this phase avoids heavy theoretical calculus. Instead, it utilizes code-driven, programmatic explanations to build a conceptual understanding of linear algebra and gradients.

The definitive resource for this phase is Andrej Karpathy's comprehensive "Neural Networks: Zero to Hero" series. The curriculum requires the practitioner to rebuild modern deep neural networks entirely from scratch, using Python to explicitly spell out every underlying mathematical operation.

The sequence begins with the micrograd module, where the engineer implements a tiny, scalar-valued autograd engine and builds a neural network library on top of it. By manually writing the backward pass (backpropagation) for simple addition and multiplication operations, the student develops a visceral, code-based understanding of how gradients flow backward through a computational graph to update model weights. This programmatic approach satisfies the need for conceptual math understanding without relying on closed-form calculus equations.

The curriculum then transitions into language modeling through the makemore series. The practitioner is formally introduced to the PyTorch framework, specifically mastering the

manipulation of multi-dimensional arrays (torch.Tensor). Understanding tensor shapes, memory storage, and broadcasting is vital, as poorly optimized tensor operations are the primary cause of latency in production systems. The engineer will incrementally build more complex architectures, starting from a bigram character-level model, advancing to a Multi-Layer Perceptron (MLP), and exploring the mechanics of activation functions (like Tanh) and Batch Normalization. The course emphasizes becoming a "Backprop Ninja," manually tracing gradients through complex layers to build an intuitive diagnostic capability.

The climax of this semester is the deconstruction of the Generatively Pretrained Transformer (GPT) architecture. Based on the seminal "Attention is All You Need" paper, the engineer will manually implement self-attention mechanisms and transformer blocks in code. Finally, the semester concludes with an exhaustive study of tokenization. The engineer will build the Byte Pair Encoding (BPE) algorithm from scratch, mimicking the tokenizer used in OpenAI's GPT series. Understanding that the tokenizer is a completely independent pipeline stage is critical for a systems engineer, as many bizarre LLM behaviors, input errors, and security vulnerabilities (such as prompt injection vectors) trace directly back to tokenization quirks.

Semester 4 Curriculum Block	**Technical Focus**	**Core Concepts**	**Primary Resource**
Autograd Engines	Manual backpropagation, gradient descent	Computational Graphs	Karpathy: micrograd
PyTorch Internals	Tensor manipulation, memory allocation	torch.Tensor operations	Karpathy: makemore (Part 1-2)
Deep Architectures	Activation functions, Batch Normalization	Multi-Layer Perceptrons	Karpathy: makemore (Part 3-5)
Transformer Logic	Self-attention, autoregressive generation	GPT Architecture	Karpathy: Let's build GPT
Tokenization	Byte Pair Encoding (BPE), text translation	Encoding/Decoding Pipelines	Karpathy: Let's build the Tokenizer

The capstone project for Semester 4 requires the engineer to construct, train, and export a scaled-down, custom autoregressive character-level transformer model. Utilizing a localized,

domain-specific dataset (such as a corpus of technical manuals or Linux kernel source code), the engineer will write the PyTorch training loop, optimize the tensor operations for training on their local hardware, and implement a custom minBPE tokenizer. The final model weights must be serialized and exported, ready to be ingested by the serving and MLOps pipelines developed in the subsequent semesters.

Semester 5: MLOps Foundations and Continuous Integration

Having developed a functional, localized machine learning model, the practitioner quickly realizes that the Python code required to train a model constitutes a minuscule fraction of an enterprise AI system. Semester 5 bridges the gap between ad-hoc model training in Jupyter notebooks and systematic, reproducible, and automated Machine Learning Operations (MLOps). The objective is to apply the rigorous standards of traditional software DevOps to the non-deterministic world of artificial intelligence.

This semester leverages the DataTalksClub MLOps Zoomcamp and Goku Mohandas's highly regarded "Made With ML" curriculum. The central tenet of MLOps is achieving total reproducibility and automation across the machine learning lifecycle.

The curriculum begins with the critical discipline of experiment tracking and model management. When a data scientist iterates on model architectures, adjusts learning rates, or alters feature sets, tracking which specific configuration produced which statistical result is paramount. The engineer will master MLflow, an open-source platform, to systematically log metadata, hyperparameters, training metrics, and resulting artifacts. This establishes a centralized Model Registry, allowing the team to version control models and cleanly transition them through staging, production, and archived environments.

Simultaneously, the practitioner must learn to version control massive datasets and model weights. Traditional Git is incapable of handling terabytes of binary data. The curriculum introduces tools like Data Version Control (DVC), allowing the engineer to version massive datasets alongside the code that processes them. This ensures that any historical training run can be perfectly reconstructed, a strict requirement for regulatory compliance and robust system debugging.

The semester then advances to workflow orchestration specifically tailored for machine learning pipelines. Building upon the scheduling concepts learned in Semester 1, the engineer will utilize frameworks like Apache Airflow, Prefect, or Kubeflow Pipelines to construct directed acyclic graphs (DAGs). These pipelines automate the sequential execution of data extraction, preprocessing, distributed model training, and evaluation. Finally, Continuous Integration and Continuous Deployment (CI/CD) practices are implemented using GitHub Actions. The practitioner will write automation scripts that automatically trigger testing suites, validation checks, and cloud infrastructure updates whenever new data is pushed or model code is modified.

Semester 5	Technical Focus	Core Technologies	Primary Resource

Curriculum Block			
Experiment Tracking	Metric logging, parameter tracking, artifacts	MLflow	MLOps Zoomcamp (Mod 2)
Model Registries	Version control, environment staging	MLflow Model Registry	MLOps Zoomcamp (Mod 2)
Data Versioning	Managing large binaries, reproducibility	DVC (Data Version Control)	GitHub MLOps Roadmaps
ML Pipeline Orchestration	Automated training graphs, DAGs	Prefect, Kubeflow	MLOps Zoomcamp (Mod 3)
CI/CD for ML	Automated testing, deployment triggers	GitHub Actions	Made With ML

The capstone project requires taking the custom transformer model developed in Semester 4 and fully operationalizing its training lifecycle. The engineer will build an automated CI/CD pipeline where a push to a specific GitHub branch triggers a GitHub Action. This action will pull the heavily versioned training data via DVC, execute the PyTorch training script, log all real-time performance metrics to a remote MLflow tracking server, and dynamically register the resulting model into the production registry only if its validation loss surpasses a predefined baseline metric. This project guarantees that the model training process is entirely hands-free and reproducible.

Semester 6: High-Performance Model Serving and Hardware Optimization

Deploying a machine learning model entails exposing its predictive capabilities to downstream systems. However, merely wrapping a complex neural network in a basic Python Flask API is grossly insufficient for production workloads. High concurrency will rapidly exhaust system memory and block CPU threads. Semester 6 dives deep into the architecture of high-performance inference servers, focusing on maximizing request throughput, minimizing response latency, and optimizing GPU/CPU hardware utilization. This semester leverages the user's existing Unraid home server environment, utilizing Docker and GPU passthrough to

emulate enterprise deployment strategies.
The curriculum focuses heavily on two open-source giants in the serving ecosystem: NVIDIA's Triton Inference Server and the vLLM serving engine.
For traditional deep learning models, embeddings, and computer vision architectures, Triton Inference Server is the undisputed industry standard. The engineer will learn to deploy models trained across various computational frameworks (PyTorch, TensorFlow, TensorRT, ONNX) into a unified serving environment without writing custom API code. A critical concept to master is dynamic batching. The engineer will configure the server to artificially delay incoming requests by a few milliseconds, grouping them into a single, massive batch before passing them to the GPU. This drastically increases hardware utilization and overall throughput, maximizing the return on expensive compute resources.
When shifting focus to generative Large Language Models (LLMs), the architectural requirements change radically due to the autoregressive, token-by-token nature of text generation. The engineer must master vLLM, a state-of-the-art serving engine specifically optimized for LLM inference. The profound engineering insight required here is understanding the memory bottleneck caused by the Key-Value (KV) cache during token generation. vLLM solves this memory fragmentation using PagedAttention, an algorithm inspired by traditional operating system virtual memory management. By fragmenting the KV cache into non-contiguous physical blocks, vLLM nearly eliminates memory waste, allowing for massive concurrent request processing. Furthermore, setting up an OpenAI-compatible API server using vLLM ensures that existing frontend applications and multi-agent frameworks can interface with locally hosted open-source models seamlessly.

Semester 6 Curriculum Block	**Technical Focus**	**Core Technologies**	**Primary Resource**
Standard Serving Engines	Multi-framework support, concurrent execution	Triton Inference Server	Triton Tutorials
Hardware Optimization	Dynamic batching, memory coalescing	Triton, ONNX, TensorRT	Triton Conceptual Guides
LLM Serving Architectures	PagedAttention, continuous batching	vLLM	vLLM Documentation
API Emulation	Drop-in replacements for proprietary APIs	vLLM OpenAI-Compatible Server	vLLM Quickstart

The Semester 6 project is the deployment of a high-performance, dual-engine serving architecture on the practitioner's Unraid server. The engineer will deploy a containerized Triton Inference Server instance, hosting an embedded representation model (such as BERT) exported to ONNX format, explicitly tuned with dynamic batching. Concurrently, they will deploy a highly quantized, open-source LLM (such as Llama-3 8B) utilizing vLLM. Finally, the engineer must write an asynchronous Python benchmarking script that blasts both servers with thousands of concurrent requests, measuring latency, token throughput, and memory utilization to empirically prove optimal server configuration.

Semester 7: Vector Search, RAG, and Advanced Information Retrieval

Modern AI applications rarely rely solely on the static knowledge embedded within a model's parameterized weights. The prevailing architecture for building intelligent, context-aware systems is Retrieval-Augmented Generation (RAG). Semester 7 focuses on the specialized infrastructure required to efficiently store, index, and retrieve massive amounts of unstructured data using vector databases.

The curriculum utilizes the Qdrant Essentials Course and DataTalksClub's LLM Zoomcamp to build deep competency in vector search mechanisms. The engineer must understand how text, images, and audio are mathematically converted into high-dimensional embeddings and mapped into a shared vector space, allowing for rapid similarity comparisons.

A deep dive into the underlying algorithms of vector databases is required. The practitioner must understand the trade-offs between exact nearest neighbor search (which is slow but perfectly accurate) and Approximate Nearest Neighbor (ANN) algorithms, specifically the Hierarchical Navigable Small World (HNSW) indexing method. Understanding these graph-based algorithms allows the engineer to tune the database parameters for either maximum recall (accuracy) or minimum latency (speed) based on the strict requirements of the application.

The curriculum evaluates different vector databases to understand their architectural nuances. For example, Milvus offers immense horizontal scalability and a distributed architecture suitable for billion-scale datasets, making it the choice for massive enterprises like Reddit. Conversely, Qdrant balances high-performance Rust-based execution with exceptional payload filtering capabilities. Qdrant's ability to combine dense vector similarity search with metadata payload filtering (e.g., filtering search results by a specific geographic location or a timestamp before calculating vector distance) is crucial for building highly accurate RAG pipelines. Furthermore, the phase introduces hybrid search—combining traditional sparse keyword search algorithms (like BM25) with dense vector embeddings to dramatically improve retrieval accuracy.

Semester 7 Curriculum Block	Technical Focus	Core Technologies	Primary Resource

Vector DB Fundamentals	Similarity metrics (Cosine, Dot Product), embeddings	Qdrant	Qdrant Essentials (Days 0-2)
Indexing Algorithms	HNSW, performance tuning, ANN search	Qdrant / Milvus	Qdrant Essentials (Day 4)
Advanced Retrieval	Hybrid search, payload filtering, re-ranking	Sparse/Dense Vectors	Qdrant Essentials (Day 3)
RAG Architecture	Ingestion pipelines, context window management	LangChain, LlamaIndex	LLM Zoomcamp

The capstone project for Semester 7 is the construction of a production-grade, containerized RAG application. The engineer will build a Python-based automated ingestion pipeline that reads a vast repository of localized documentation, chunks the text using semantic boundaries, generates embeddings utilizing the Triton-hosted BERT model from Semester 6, and upserts them into a local Qdrant collection alongside rich metadata payloads. They will then build a querying API that takes user questions, performs a hybrid search (vector similarity combined with metadata filtering), and feeds the precisely retrieved context into the vLLM server to generate accurate, hallucination-free responses.

Semester 8: Autonomous Multi-Agent Orchestration and Production Observability

The final semester of the curriculum elevates the AI system from a passive, single-turn question-answering tool into an autonomous, goal-oriented architecture. Simultaneously, it implements the critical observability required to keep these complex, non-deterministic systems healthy in a live production environment.

The first half of the semester explores advanced Multi-Agent Systems, directly leveraging the user's prior exposure to build enterprise-ready configurations. Utilizing courses from DeepLearning.AI, the engineer will study frameworks like CrewAI and LangGraph. The paradigm shifts from executing single, linear prompts to orchestrating multiple specialized AI agents that can plan, collaborate, utilize external APIs, and autonomously execute complex business workflows.

CrewAI allows the practitioner to define agents with highly specific roles, backstories, and

available tools, orchestrating them to perform multi-step tasks such as automated technical research or financial analysis. LangGraph introduces advanced graph-based state management, enabling cyclic workflows where agents can reflect on their previous outputs, dynamically correct errors, and maintain long-term memory across sessions. Understanding these frameworks is essential for building AI applications that integrate seamlessly across varying software platforms.

The second half of the semester is arguably the most critical for an MLOps specialist: Production Monitoring and Observability. Operating autonomous agents and generative models introduces high degrees of unpredictability. The engineer must monitor not only traditional system metrics—CPU load, GPU memory consumption, and network latency—but also model-specific metrics.

Utilizing Prometheus for time-series metric collection and Grafana for dashboard visualization, the engineer will instrument their entire technology stack. Furthermore, they will utilize tools like Evidently AI to monitor data health and trigger alerts when the distribution of incoming requests deviates significantly from the training baseline (detecting data drift) or when the generative outputs display toxicity or high hallucination rates.

Semester 8 Curriculum Block	**Technical Focus**	**Core Technologies**	**Primary Resource**
Multi-Agent Orchestration	Role definition, tool integration, collaboration	CrewAI	DeepLearning.AI: CrewAI
State Management	Cyclic workflows, memory, graph routing	LangGraph	DeepLearning.AI: LangGraph
System Observability	Time-series metrics, dashboarding	Prometheus, Grafana	MLOps Zoomcamp (Mod 5)
ML Monitoring	Drift detection, output evaluation	Evidently AI	MLOps Zoomcamp (Mod 5)

The final capstone project requires the synthesis of the entire four-year curriculum. The engineer will deploy a fully observable, multi-agent autonomous research system on their Unraid server. A frontend interface will communicate with a FastAPI backend (Semester 2), which routes complex inquiries to a LangGraph agent supervisor (Semester 8). The supervisor will delegate tasks to worker agents built with CrewAI (Semester 8) that query the Qdrant vector database (Semester 7) for internal knowledge and hit external APIs for live data. The

agents will generate logic via the locally hosted vLLM instance (Semester 6), powered by a model tracked via MLflow (Semester 5). The entire infrastructure will be containerized and orchestrated (Semester 1), with real-time hardware metrics and model-drift statistics streamed to a comprehensive Grafana dashboard (Semester 8).

Free & Industry-Accepted Certifications

As you progress through this curriculum, you can validate your skills and build a robust portfolio by acquiring the following free, industry-recognized certificates and badges:

Anthropic Academy

Anthropic recently launched a completely free training academy (hosted on Skilljar) that provides certificates upon completion. Highly relevant certifications for an AI Systems Engineer include:

- **Building with the Claude API:** Validates skills in full-spectrum API development and integrating Anthropic models into custom architectures.
- **Introduction to Model Context Protocol (MCP):** A crucial, cutting-edge certification for building MCP servers and clients in Python to seamlessly connect Claude with external APIs and services.
- **Claude Code in Action:** Covers integrating Claude Code into your development workflow and steering multi-step autonomous workflows securely.

Open-Source Platform & Tooling Certifications

- **DataTalksClub Zoomcamp Certificates:** By completing the capstone projects and peer-review processes for the Data Engineering, MLOps, and LLM Zoomcamps, you earn highly respected, completely free industry certificates. [1]
- **Qdrant Essentials Certification:** Passing the final quiz after building a production-grade documentation search engine grants you an official certification from Qdrant.
- **Hugging Face Certificates:** Hugging Face offers free Certificates of Completion (for passing 80% of assignments) and Certificates of Excellence (for 100% completion) for their Deep Reinforcement Learning and AI Agents open-source courses.
- **Databricks Fundamentals Accreditation:** A free, non-proctored 20-question assessment that awards a verifiable badge for foundational knowledge of the Databricks Data Intelligence and Lakehouse platform.

Cloud & Big Tech Credentials

- **Google Cloud AI & ML Learning Path:** Google offers completely free cloud labs and learning paths covering generative AI and ML workflows, acting as a great on-ramp for their cloud ecosystem.
- **IBM AI Foundations (via SkillsBuild):** Provides free Credly badges that you can display on LinkedIn to validate your foundational AI and model structure knowledge.
- **OpenAI Academy:** OpenAI's free learning hub provides workshops and is rolling out free certifications ranging from foundational AI fluency to prompt engineering and developer

operations.

Step-by-Step Execution Guide & Progress Checklist

This section provides a granular, actionable breakdown of the curriculum. Check off these modules, quizzes, and specific homework assignments as you progress to stay on track.

Semester 1 Checklist: Infrastructure & Data Engineering

- [ ] **MIT "Missing Semester" (OS/Linux Tooling):**
  - [ ] Watch the Jan 12th "Course Overview + Introduction to the Shell" lecture.
  - [ ] Complete the "Version Control and Git" exercises.
  - [ ] Watch the "Debugging and Profiling" lecture and complete associated labs.
- [ ] **Data Engineering Zoomcamp (DataTalksClub):**
  - [ ] Complete **Module 1**: Set up Docker and PostgreSQL locally; configure GCP using Terraform. Submit Homework 1.
  - [ ] Complete **Module 2**: Deploy the Kestra orchestration engine and schedule your first data pipeline. Submit Homework 2.
  - [ ] Complete **Module 3**: Load data into BigQuery and practice partitioning/clustering. Submit Homework 3.
  - [ ] Complete **Module 4**: Write analytics engineering transformations using dbt. Submit Homework 4.
  - [ ] Complete **Modules 5 & 6**: Process data batches using Apache Spark and experiment with real-time streaming using Apache Kafka and PyFlink.
- [ ] **Capstone 1:** Build an end-to-end data pipeline on your Unraid server pulling a public API, storing in Postgres, and orchestrating via Kestra.

Semester 2 Checklist: APIs & Microservices

- [ ] **freeCodeCamp FastAPI Crash Course:**
  - [ ] Set up a Python environment and write your first API endpoint.
  - [ ] Complete the sections on Path Parameters, Query Parameters, and combining both.
  - [ ] Master HTTP Methods: Implement POST (with request bodies), PUT, and DELETE methods.
- [ ] **Database & Testing Integration:**
  - [ ] Connect your FastAPI app to a MongoDB database using PyMongo. Write scripts to parse and insert data.
  - [ ] Containerize your API using Docker.
- [ ] **Asynchronous Microservices:**
  - [ ] Configure a local Redis instance and connect it to FastAPI.
  - [ ] Implement Python asyncio background tasks that push work into Redis Streams.
- [ ] **Capstone 2:** Deploy a multi-container Docker Compose setup containing a FastAPI gateway, MongoDB, a Redis message broker, and a background Python worker script.

Semester 3 Checklist: System Architecture

- [ ] **Stanford CS329S (Machine Learning Systems Design):**
  - [ ] Read Week 1 notes: "ML in research vs. ML in production" and "ML systems vs. traditional software".
  - [ ] Read Week 3 notes: "Data management" and the different layers of the data pipeline.
- [ ] **Full Stack Deep Learning (FSDL):**
  - [ ] Complete Lab 4: Experiment Management.
  - [ ] Complete Lab 5: Troubleshooting & Testing.
  - [ ] Complete Lab 8: Model Monitoring.
- [ ] **Capstone 3:** Write a detailed System Architecture Document defining latency constraints, infrastructure routing, and continuous monitoring pipelines for a theoretical fraud detection engine.

Semester 4 Checklist: Neural Networks (Zero to Hero)

- [ ] **Part 1 (micrograd):** Code a tiny autograd engine. Manually trace the backward pass for addition, multiplication, and a single artificial neuron.
- [ ] **Part 2 (makemore - Bigram):** Complete exercises E01-E06. Ensure you learn to use F.cross_entropy instead of manual calculation.
- [ ] **Part 3 (makemore - MLP):** Complete exercises E01-E03. Practice tuning hyperparameters and initializations to beat the baseline validation loss.
- [ ] **Part 4 (Becoming a Backprop Ninja):** Do not skip this! Work through the 4 exercises to manually write the backward pass for cross-entropy, BatchNorm, and linear layers without loss.backward().
- [ ] **Part 7 (GPT Tokenizer):** Build the BPE algorithm. Implement the encode() and decode() functions from scratch.
- [ ] **Capstone 4:** Train your custom autoregressive model, optimize its tensor operations on your local hardware, and export the weights.

Semester 5 Checklist: MLOps Foundations

- [ ] **MLOps Zoomcamp (DataTalksClub):**
  - [ ] Complete **Module 2**: Instrument your Python training script with MLflow to track parameters and metrics. Register a model. Submit Homework 2.
  - [ ] Complete **Module 3**: Build an automated training DAG using Prefect or Kubeflow. Submit Homework 3.
  - [ ] Complete **Module 4**: Deploy the tracked model via a batch job or web service. Submit Homework 4.
  - [ ] Complete **Module 5**: Implement drift detection utilizing Evidently AI. Submit Homework 5.
- [ ] **CI/CD Integration:**
  - [ ] Use Data Version Control (DVC) to manage large dataset binaries.

    - [ ] Set up GitHub Actions to trigger automated tests upon pushing new code.
- [ ] **Capstone 5:** Build a fully automated GitHub Action pipeline that pulls data via DVC, trains your Semester 4 model, logs to MLflow, and registers the artifact.

Semester 6 Checklist: Model Serving

- [ ] **Triton Inference Server (NVIDIA):**
    - [ ] Create a local model repository structure and fetch example models.
    - [ ] Launch the Triton Inference Server via Docker container and send your first HTTP/gRPC inference request.
    - [ ] Configure Triton's config.pbtxt to enable dynamic batching.
- [ ] **vLLM Engine (LLMs):**
    - [ ] Install vLLM and test Offline Batched Inference.
    - [ ] Launch the vLLM OpenAI-Compatible API Server.
    - [ ] Query the server locally using the standard OpenAI Python client package.
- [ ] **Capstone 6:** Deploy both Triton (for an ONNX embedding model) and vLLM (for an open-source LLM) on your Unraid server. Write a Python script to hit both servers with concurrent traffic and benchmark token latency.

Semester 7 Checklist: Vector DBs & RAG

- [ ] **Qdrant Essentials Course:**
    - [ ] **Day 0:** Initialize the client, create a collection, insert points, and perform a basic similarity search.
    - [ ] **Day 3:** Follow the demo to implement Hybrid Search, utilizing sparse keyword vectors (BM25) alongside dense embeddings.
    - [ ] **Day 6 (Final Project):** Build a production-ready documentation search engine utilizing chunking, payloads, and reranking.
    - [ ] **Certification:** Take and pass the Qdrant Essentials Certification quiz.
- [ ] **LLM Zoomcamp (DataTalksClub):**
    - [ ] Complete **Module 1**: Introduction to RAG pipelines. Submit Homework 1.
    - [ ] Complete **Module 2**: Vector search and semantic retrieval. Submit Homework 2.
- [ ] **Capstone 7:** Build a containerized RAG application that chunks documentation, embeds it via your Triton server, upserts to Qdrant, and generates answers using vLLM.

Semester 8 Checklist: Agents & Observability

- [ ] **DeepLearning.AI Agent Courses:**
    - [ ] Take the "Multi AI Agent Systems with crewAI" course. Configure agents with distinct roles to perform a multi-step research task.
    - [ ] Take the "AI Agents in LangGraph" course. Build cyclic workflows that utilize long-term memory and self-correction.
- [ ] **Observability Stack:**
    - [ ] Deploy Prometheus to collect time-series hardware metrics from your Unraid server and Docker containers.

    - [ ] Deploy Grafana and build a dashboard to visualize CPU, GPU, and network latency.
- [ ] **Capstone 8 (Final Portfolio Project):** Orchestrate a LangGraph/CrewAI multi-agent system behind your FastAPI gateway. The agents must search Qdrant for context, infer utilizing vLLM, and all system/model health metrics must stream to a live Grafana dashboard.

Works cited

1. Data Engineering Zoomcamp: Free Data Engineering Course and Certification - DataTalks.Club, accessed June 2, 2026, https://datatalks.club/blog/data-engineering-zoomcamp.html
2. Free DataTalks.Club Courses: ML, Data Engineering, MLOps, LLM & AI Dev Tools Zoomcamps, accessed June 2, 2026, https://datatalks.club/blog/guide-to-free-online-courses-at-datatalks-club.html